Звіт

Сортування і структури даних

Якубовської Антоніни

Задача 1 (варіант V5) - база студентів із трьома операціями, три реалізації та їх порівняння

Задача 2 (варіант S5) - сортування бази за полем m_group.

Дані: data/students.csv, 100 000 записів: ім'я, прізвище, email, дата народження, група, рейтинг, телефон

Операції та їх частота (співвідношення 2 : 100 : 30): Op1 - 100 студентів із найвищим рейтингом, Op2 - оновлення рейтингу за email, Op3 - групи з найбільшим середнім рейтингом.

Задача I: три реалізації бази

Усі варіанти реалізують однаковий інтерфейс (init, get_top_100, update_rating, get_best_groups) і відрізняються структурами даних. Варіанти B і C підтримують індекси постійно: упорядкований multiset<(рейтинг, email)> для Top-100 та агрегати GroupStats (сума рейтингів + кількість) для кожної групи. У A індексу рейтингу немає, тому кожен запит проходить усіх записів.

Варіант	Структури	Op1: Top-100	Op2: оновлення рейтингу	Op3: найкраща група
A	vector<Student> + unordered_map<email, індекс>	O(N log 100): partial_sort по всіх N	O(1) в середньому	O(N): перерахунок усіх груп кожного разу
B	map<email, Student> + multiset + map<група, GroupStats>	O(100 · log N)	O(log N)	O(G), G = кількість груп
C	unordered_map<email, Student> + multiset + unordered_map<група, GroupStats>	O(100) кроків + 100 пошуків у хеш-таблиці	O(log N) через multiset, пошук за email O(1)	O(G)

При оновленні рейтингу B і C видаляють старий запис з multiset, вставляють новий і коригують суму групи на різницю (O(1)), тому Op3 не потребує перерахунку по записах. Усі три варіанти повертають 100 студентів у Top-100 і одну й ту ж найкращу групу

Масштабування варіанту C - Найповільніша операція варіанту C - Op3 її час описується функцією $T(n) = 8{,}41 \cdot 10^{-3} \cdot G(n) + 4{,}04 \cdot 10^{-5} \cdot n$ з $R^2 = 0{,}9965$, де G(n) — кількість груп. Op1 залишається приблизно на рівні 17–20 мкс, Op2 зростає з 0,27 до

2,5 мкс. зважений по 2 : 100 : 30 середній час операції на 100 000 записів ≈ 11,0 мкс, а

бенчмарк дав ≈ 11,1 мкс

Лінійна регресія для Op3

Op3 - один прохід по вузлах unordered_map груп з одним діленням на групу, тобто O(G), де $G \leq n$. Тому базисні функції: $f_1 = G(n)$ (виміряна кількість унікальних груп у таблиці), $f_2 = n$ (зростання таблиць у пам'яті, кеш-промахи), $f_3 = 1$.

Пам'ять

Виміряна пам'ять зростає лінійно: ≈ 565 байт на запис. Причина: sizeof(Student) = 184 байти, email довший за внутрішній буфер std::string і копіюється в кілька місць (ключ хеш-таблиці, поле студента, вузол multiset), додаються вузли хеш-таблиці та дерева.

Задача 2: сортування за m_group

std::sort виявився у 3,3–3,4 рази швидшим за Radix Sort з репозиторію на всіх розмірах бази, а Radix Sort за пакованим ключем обігнав std::sort у 3–7 разів. Група має формат [A-Z]{3}-[0-9]{2}, тому використання української абетки не потрібне: сортування звичайне лексикографічне.

Звичайний програє але варіант з пакованим ключем - група пакується в ціле число (три літери у системі з основою 26 і дві цифри, максимум 1 757 599), Radix Sort за 3 проходи по 8 біт сортує лише пари (ключ, індекс) по 8 байт, і лише потім одним проходом переміщує Student..

Стійкість до DoS-атак

Варіант C вразливий: unordered_map з ключем-email у libstdc++ використовує детерміновану хеш-функцію без випадкового зародка. Зловмисник може підібрати адреси з однаковим хешем, усі вони потраплять у один кошик, пошук стає O(N) замість O(1), а вставка N таких записів - $O(N^2)$. Оскільки операція оновлення рейтингу - 75 % навантаження, сервіс легко перевантажити

Висновки

Найкраща продуктивність для суміші 2 : 100 : 30 у варіанта, друга у B; A відстає через повні проходи по базі. У варіанту C найповільніша операція Op3: $T(n) = 8{,}41 \cdot 10^{-3} \cdot G(n) + 4{,}04 \cdot 10^{-5} \cdot n$ бо час виходить ≈ 38 мкс, бо груп лише ≈ 4 000. Op1 ≈ 17–20 мкс без залежності від N, Op2 зростає з 0,27 до 2,5 мкс. Для сортування об’єктів Student Radix Sort з копіюванням цілих об’єктів повільніший за std::sort у 3,3–3,4 рази; сортування пакованих ключів з одним переміщенням об’єктів у 3,2 рази швидше за std::sort на 100 000 записів. Хеш-таблиця без захисту вразлива до DoS